

Heap

1.定义

堆是一种数据结构，指的是一个无序的堆。

2.Binary Heap

binary heap 也是一个完全二叉树，只不过每个节点代表一个item

- 节点中的值满足heap-property:
- max-heap:除了根节点的节点，该节点的值都小于等于它父节点的值
- min-heap:除了根节点的节点，该节点的值都大于等于它父节点的值

可以用一个数组来表示一个binary heap

3.对Binary Max-Heap的操作

1.HeapInsert

向一个表示binary max-heap的数组中插入一个item

只须将这个item放在数组末尾

我们需要在插入item后维持heap property,沿着通往根的路径进行比较和交换

```
1 HeapInsert(A,x):
2   heap_size+=1
3   A[heap_size]:=x
4   idx:=heap_size
5   while idx>1 and A[Floor(idx/2)]<A[idx]
6       swap(A[Floor(idx/2)],A[idx])
7       idx:=Floor(idx/2)
```

运行时间为 $O(\lg n)$

2.HeapExtractMax

从一个heap中移除maximum item并返回

但是如何得到返回的heap呢？

把last item移到根节点

同样需要维持heap property

```

1 HeapExtractMax(A):
2   max_item:=A[1]
3   A[1]=A[heap_size--]
4   MaxHeapify(1,A)
5   return max_item
6
7 MaxHeapify(idx,A):
8   idx_l:=2*idx,idx_r:=2*idx+1
9   idx_max:=(idx_l<=heap_size and A[idx_l]>A[idx]) ? idx_l : idx
10  idx_max:=(idx_r<=heap_size and A[idx_r]>A[idx_max]) ? idx_r : idx_max
11  if idx_max!=idx
12     Swap(A[idx_max],A[idx])
13     MaxHeapify(idx_max,A)

```

4.Application of heaps--Priority Queue

1.Priority Queue

- Queue ADT operations:
 - add(item):add item to the queue
 - remove():remove the next item y from queue,return y
- queuing discipline:
 - FIFO Queue
 - LIFO Queue (Stack)
 - **Priority Queue**:each item associated with a priority,remove always deletes the item with max(or min) priority

2.HeapSort

```

1 HeapSort(I):
2   heap:=BuildMaxHeap(I)
3   for i:=n down to 2
4     cur_max:=heap.HeapExtractMax()
5     I[i]:=cur_max

```

- In each iteration:
- Place one item in the array to its final position
- Place max item in current heap to its final position
- Place ith biggest item to position n-i+1

运行时间为 $O(n\lg n)$

给出数组I[1,...,n],如何构造一个max-heap

```

1 BuildMaxHeap(A):
2   heap_size:=n
3   for i:=Floor(n/2) down to 1
4     MaxHeapify(i,A)

```